



Why learn with us ?



Training From Experts



One - on – One Guidance



100% Practical



Real-Time Projects & Case Studies



Placement Preparation Support



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

Contact :- 8085896740

 @ hello_world_institute

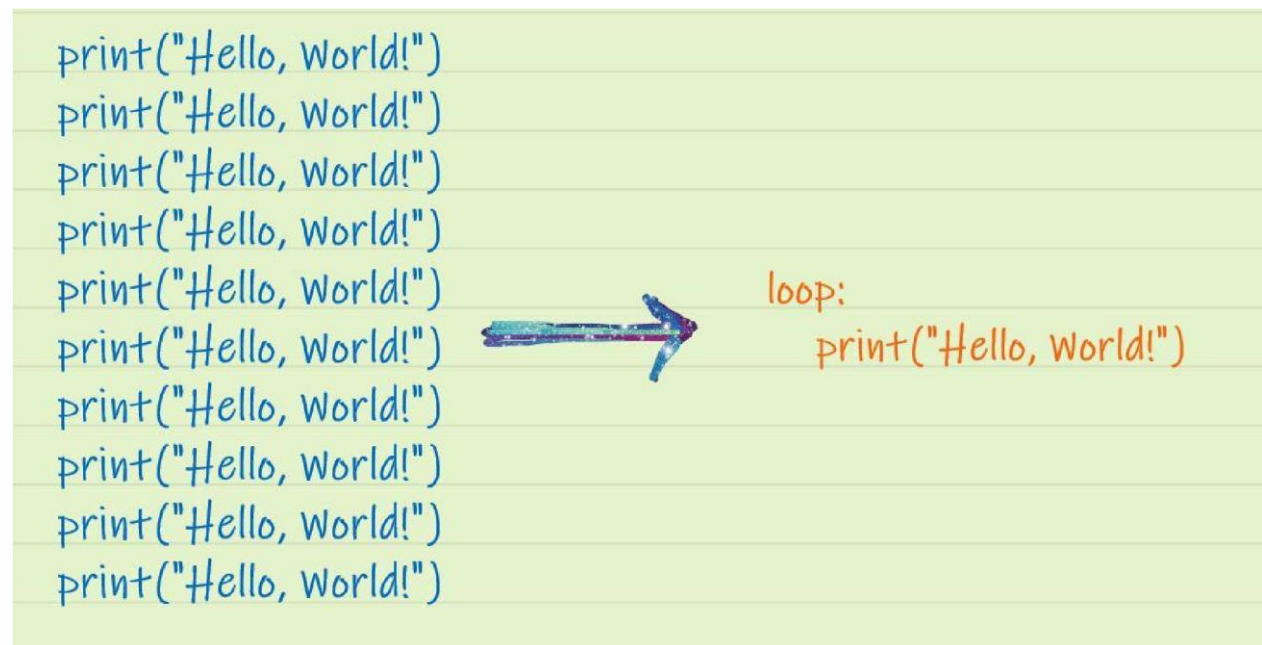
 Hello World Institute

Address : 2nd Floor , Near KFC , Tower Square , Bhawarkua , Indore

Loops and their importance:

In a programming language, a loop is a statement that contains instructions that continually repeats until a certain condition is reached.

Loops help us remove the redundancy of code when a task has to be repeated several times. With the use of loops, we can cut short those hundred lines of code to a few. Suppose you want to print the text "Hello, World!" 10 times. Rather than writing a print statement 10 times, you can make use of loops by indicating the number of repetitions needed.



In general, statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on. There may be a situation when you need to execute a block of code several number of times.

Programming languages provide various control structures that allow for more complicated execution paths.

A loop statement allows us to execute a statement or group of statements multiple times. The following diagram illustrates a loop statement –



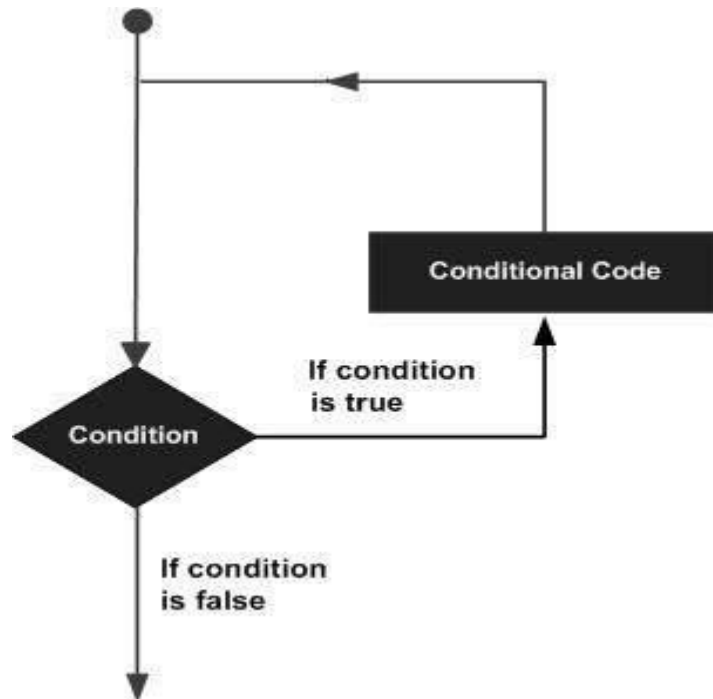
OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS



Loop Types

The three types of loops in Python programming are:

1. while loop
2. for loop
3. nested loops

While loop

It continually executes the statements (code) as long as the given condition is TRUE. It first checks the condition and then jumps into the instructions.

Syntax:

```
while condition:  
    statements(code)
```



OFFLINE CLASSES



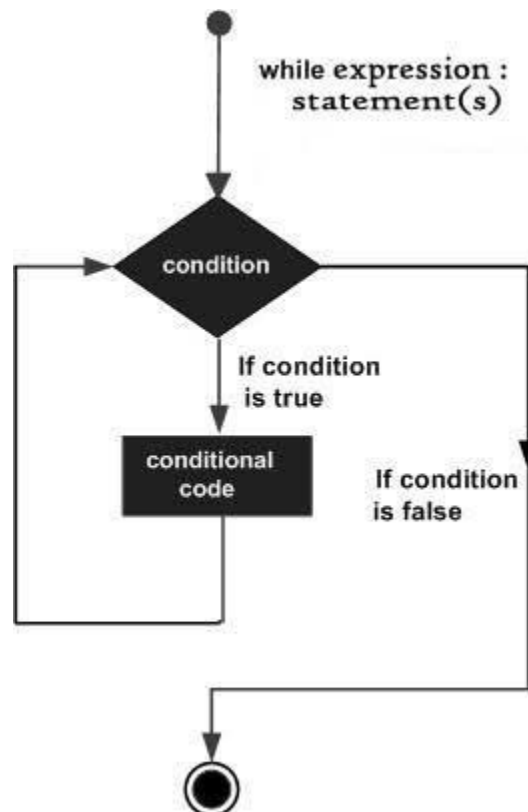
ONLINE LECTURE



RECORDED ACCESS

Inside the while loop, we can have any number of statements. The condition may be anything as per our requirement. The loop stops running when the condition fails (become false), and the execution will move to the next line of code.

Flow Diagram of while loop



It first checks the condition, executes the conditional code if the condition is TRUE, and checks the condition again. Program control exits the loop if the condition is FALSE.

For loop

A for loop is used to iterate over a sequence like lists, type, dictionaries, sets, or even strings. Loop statements will be executed for each item of the sequence.



OFFLINE CLASSES



ONLINE LECTURE

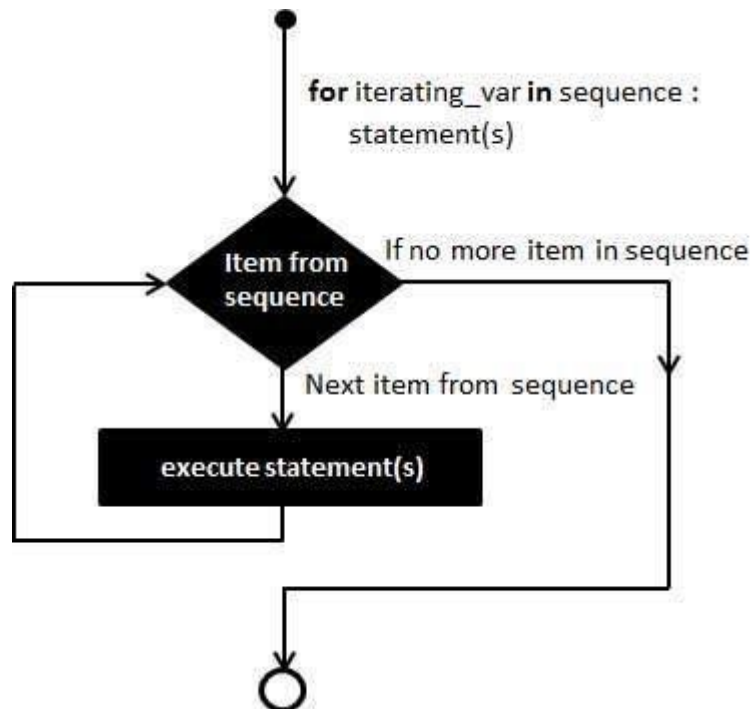


RECORDED ACCESS

Syntax of for loop:

```
for item in iterator:  
    statements(code)
```

Flow diagram of for loop:



Takes the first item of the iterable, executes the statement, and moves the pointer to the next item until it reaches the last item of the sequence.

Nested loops

Nested loops mean using a loop inside another loop. We can use any type of loop inside any type of loop. We can use a while loop inside for loop, a for loop inside a while loop, a while loop inside a while loop, or a for loop inside a for loop.



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

Nested loops

while condition:
statements

while condition:
statements

for item in iterator:
statements

while condition:
statements

for item in iterator:
statements

for item in iterator:
statements

while condition:
statements

for item in iterator:
statements

Example: Create a list of even numbers between 1 and 10

```
even_list = []
for item in range(1, 11):
    while item % 2 == 0:
        even_list.append(item)
        break
print("Even Numbers: ", even_list)
```

Loop Control Statements

Loop control statements are used to change the flow of execution. These can be used if you wish to skip iteration or stop the execution.



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

The three types of loop control statements are:

1. break statement
2. continue statement
3. pass statement

Break statement

Based on the given condition, the break statement stops the execution and brings the control out of the loop.

Example: Create a list of the odd numbers between 1 and 20 (use while, break)

```
num = 1
odd_nums = []
while num:
    if num % 2 != 0:
        odd_nums.append(num)
    if num >= 20:
        break
    num += 1
print("Odd numbers: ", odd_nums)
```

Example: Stop the execution if the current number is 5 (use for, break)

```
for num in range(1, 11):
    if num == 5:
        break
    else:
        print(num)
```



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

Continue statement

Continue statement is used to skip the current iteration when the condition is met and allows the loop to continue with the next iteration. It does not bring the control out of the loop unlike the break statement.

Example: Skip the iteration if the current number is 6 (use while, continue)

```
num = 0

while num < 10:

    num += 1

    if num == 6:

        continue

    print(num)
```

Example: Skip the iteration if the current number is 6 (use for, continue)

```
for num in range(1, 11):

    if num == 6:

        continue

    print(num)
```

Pass Statement

Pass statement is used when we want to do nothing when the condition is met. It doesn't skip or stop the execution; it just passes to the next iteration. Sometimes we



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

use comment which is ignored by the interpreter. Pass is not ignored and can be used with loops, functions, classes, etc.

It is useful when we do not want to write functionality at present but want to implement it in the future.

Example: while, pass statement

```
num = 1

while num <= 10:

    if num == 6:

        pass

    print(num)

    num += 1
```

Example: for, pass statement

```
for num in range(1, 11):

    if num == 6:

        pass

    print(num)
```



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS